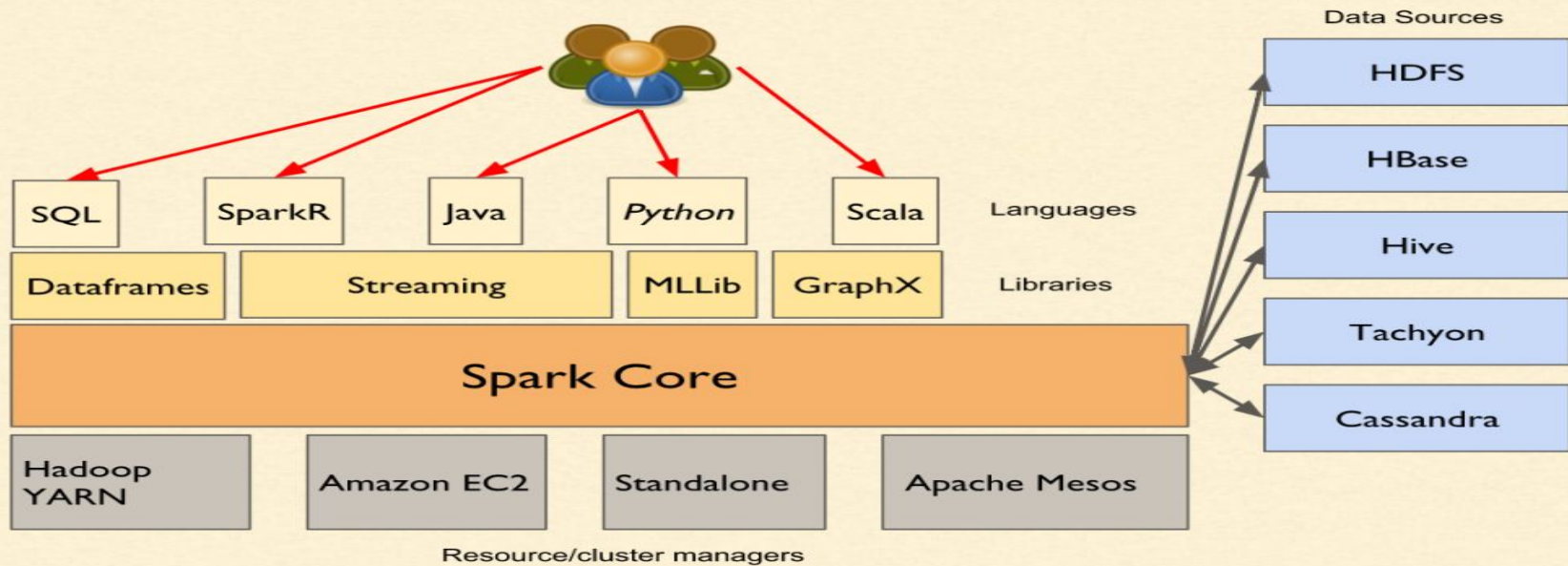


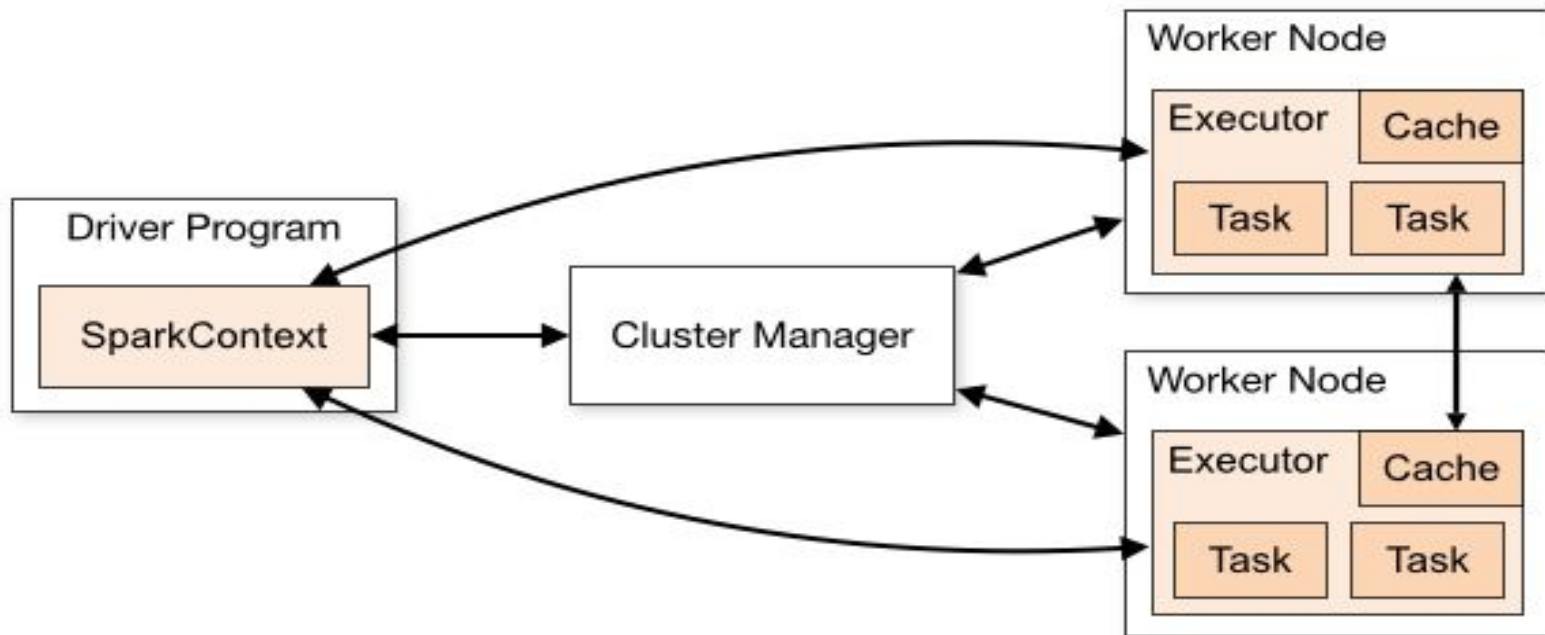
Spark Internals

components

- **Job:** A piece of code which reads some input from HDFS or local, performs some computation on the data and writes some output data.
- **Stages:** Jobs are divided into stages. Stages are classified as a Map or reduce stages (Its easier to understand if you have worked on Hadoop and want to correlate). Stages are divided based on computational boundaries, all computations (operators) cannot be Updated in a single Stage. It happens over many stages.
- **Tasks:** Each stage has some tasks, one task per partition. One task is executed on one partition of data on one executor (machine).
- **DAG:** DAG stands for Directed Acyclic Graph, in the present context its a DAG of operators.
- **Executor:** The process responsible for executing a task.
- **Driver:** The program/process responsible for running the Job over the Spark Engine
- **Master:** The machine on which the Driver program runs
- **Slave:** The machine on which the Executor program runs

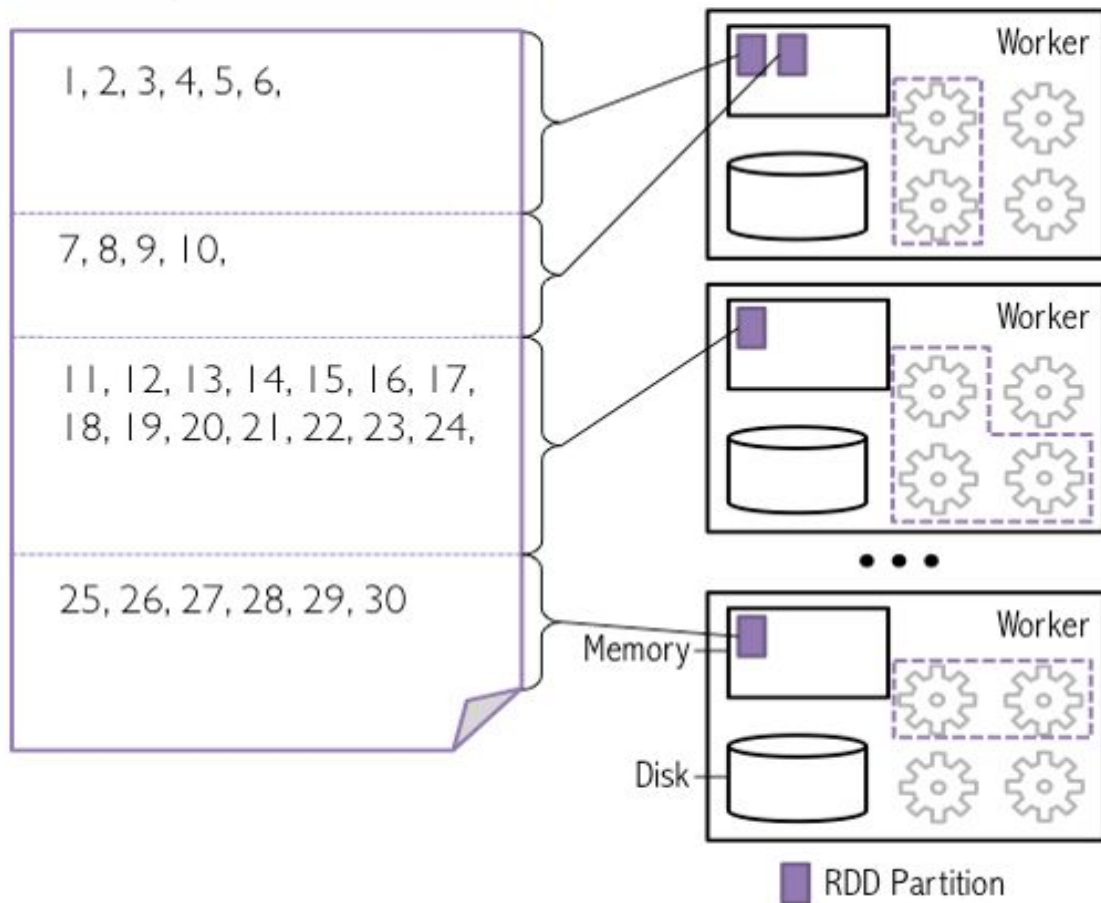
Spark Architecture





Dataset is broken into partitions

Partitions are each stored in a worker's memory



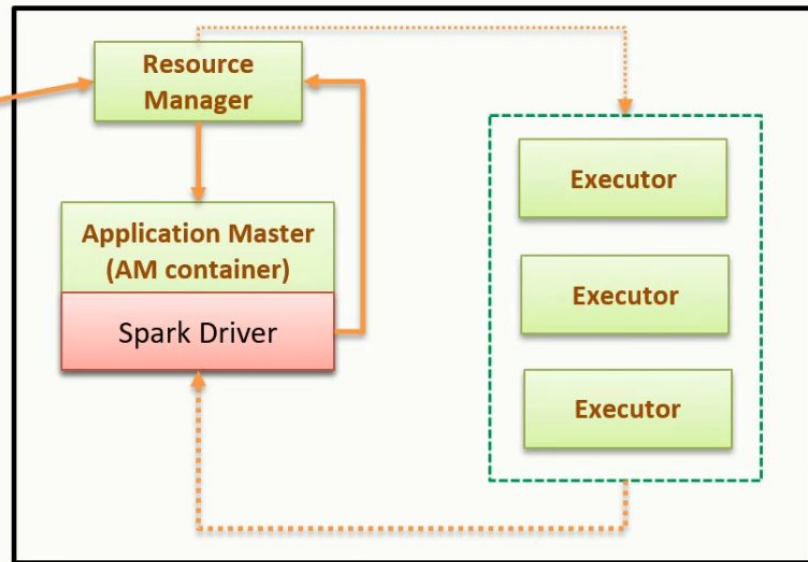


Cluster Mode

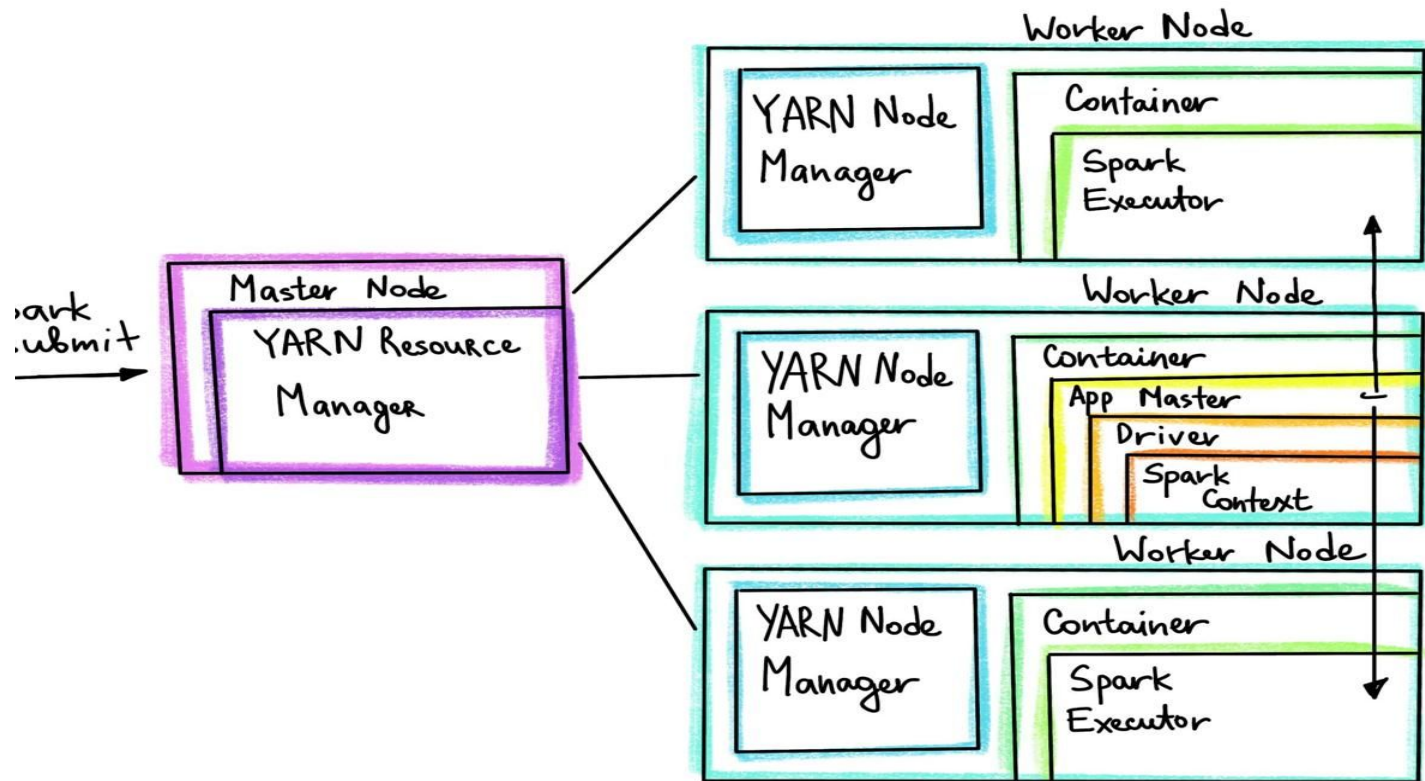


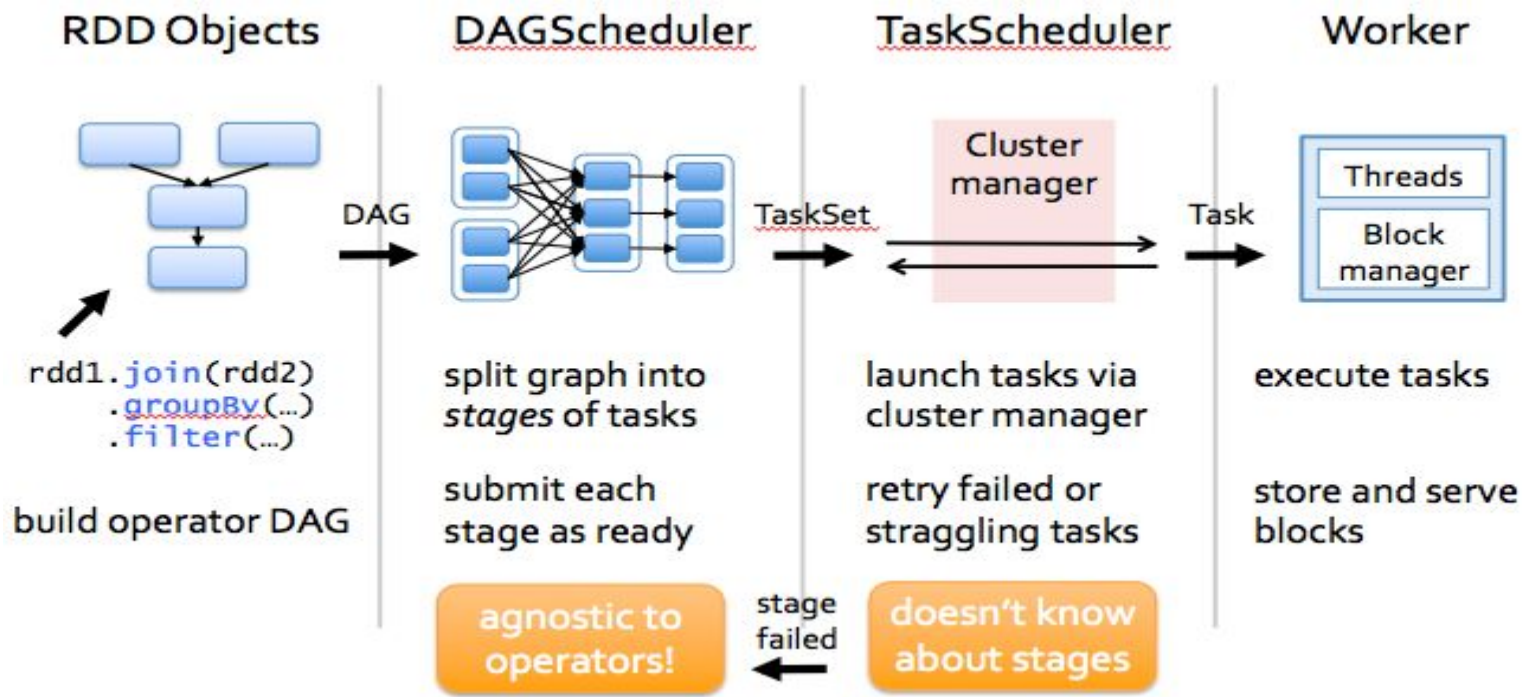
Client Machine

Spark-submit



YARN Cluster 





Datasets

RDDs

- Functional Programming
- Type-safe

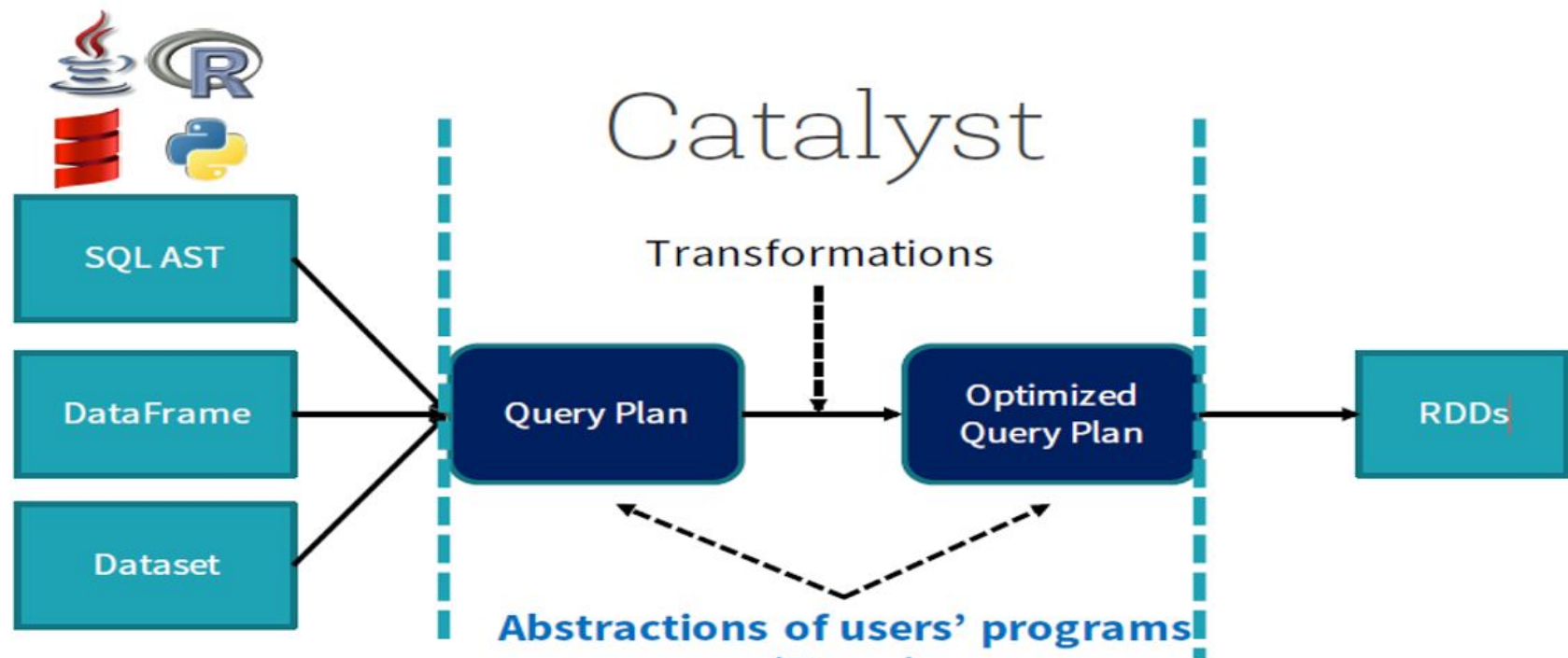
Dataframes

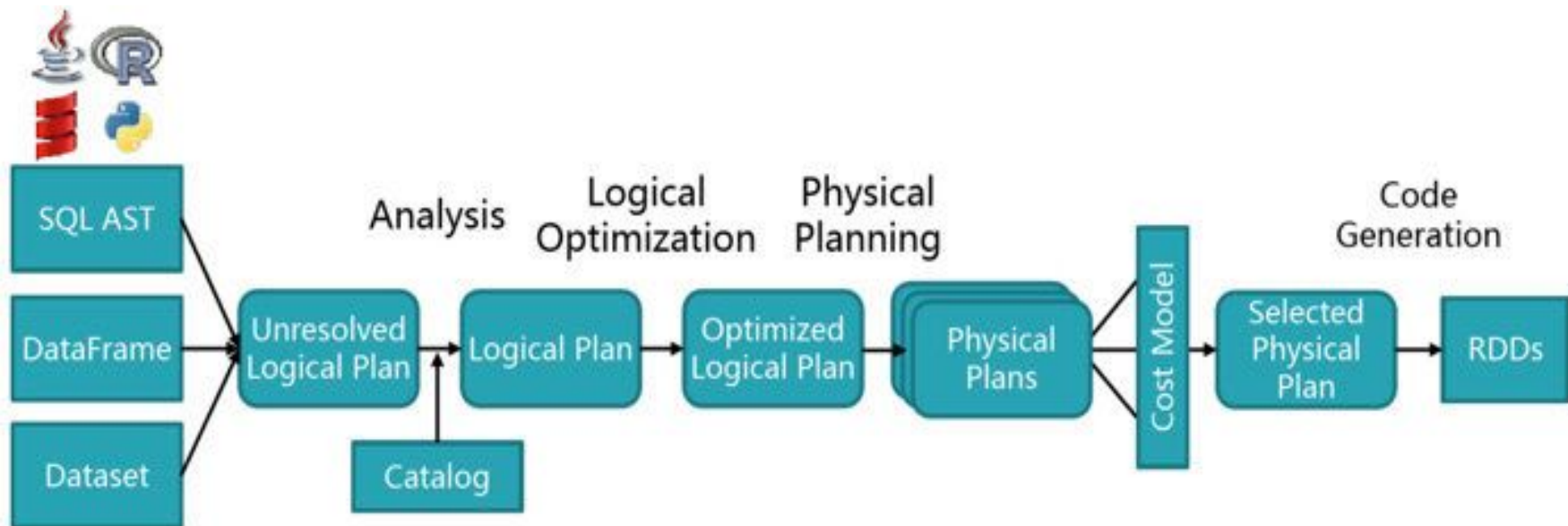
- Relational
- Catalyst query optimization
- Tungsten direct/packed RAM
- JIT code generation
- Sorting/suffling without deserializing

Encoder

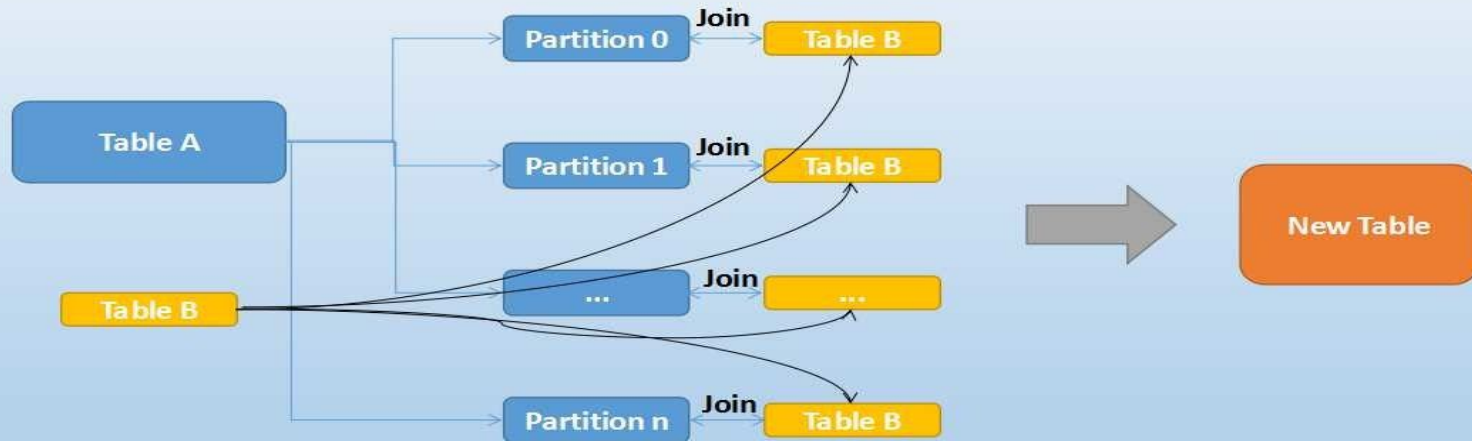
 **Spark**

How Catalyst Works: An Overview

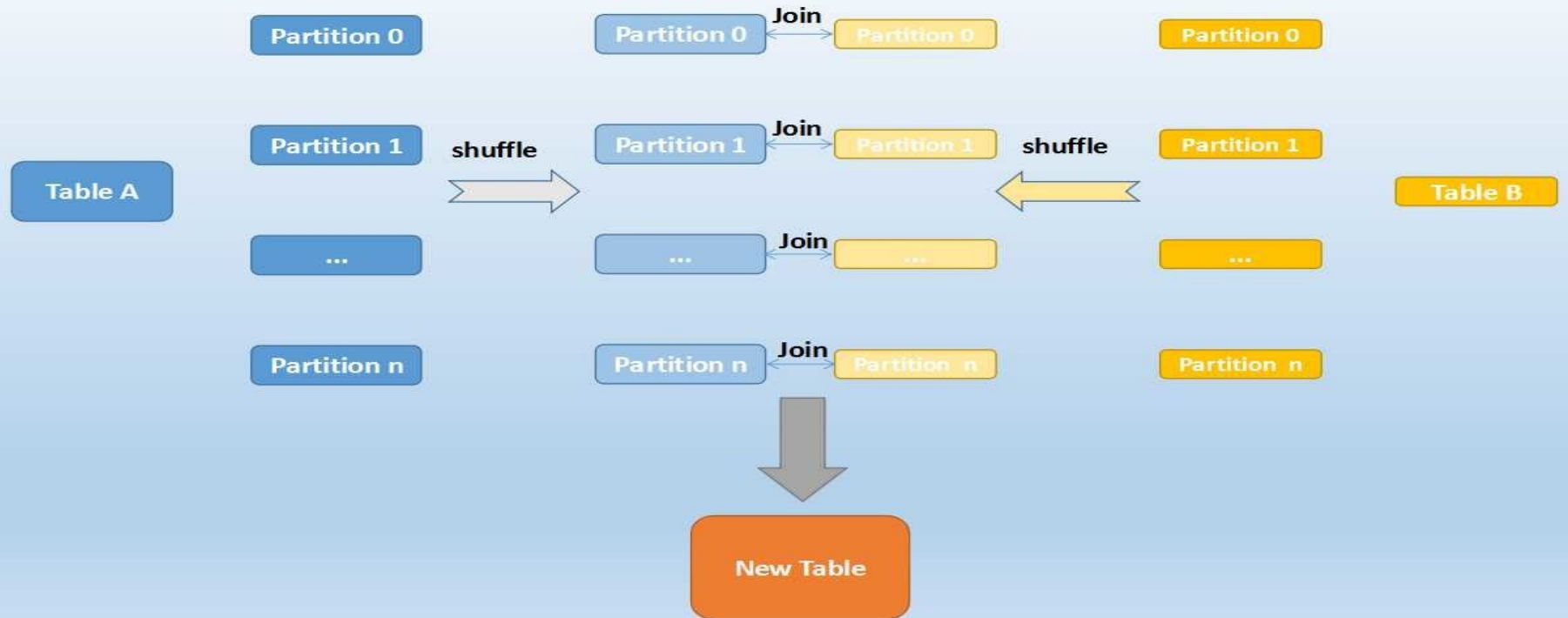




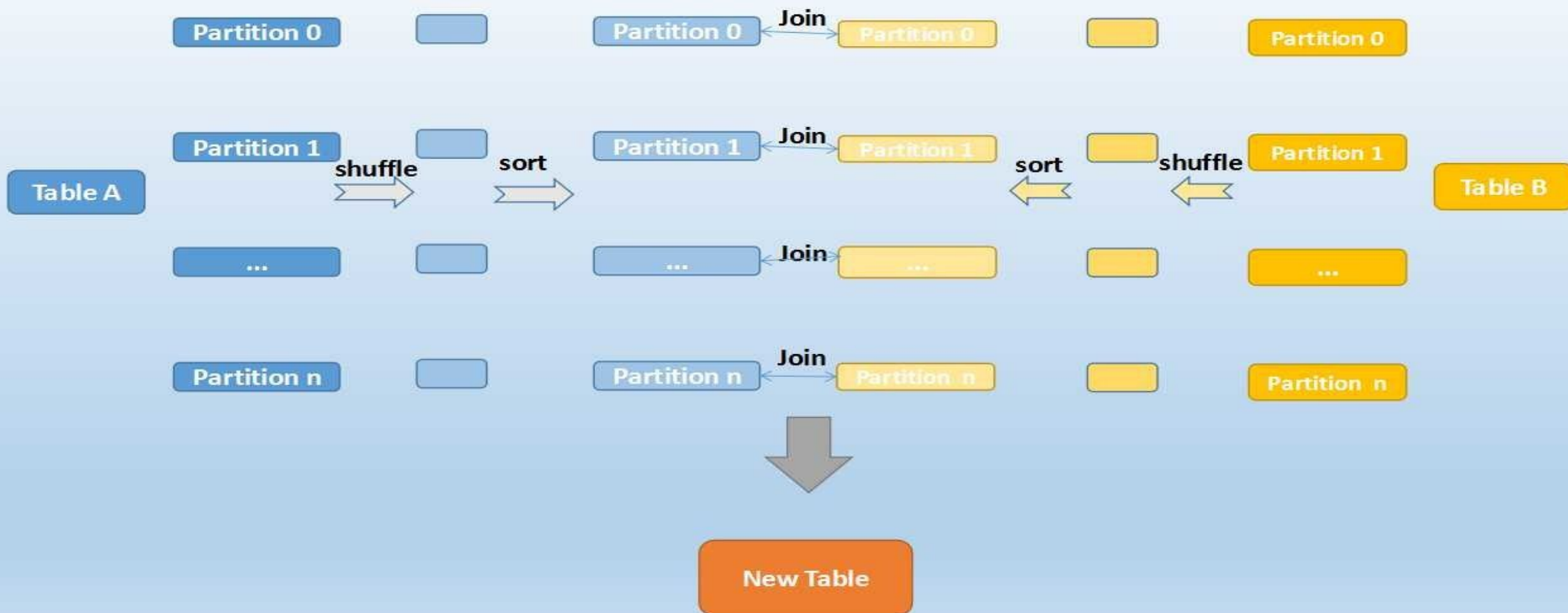
Broadcast Join in SparkSQL



Shuffle Hash Join

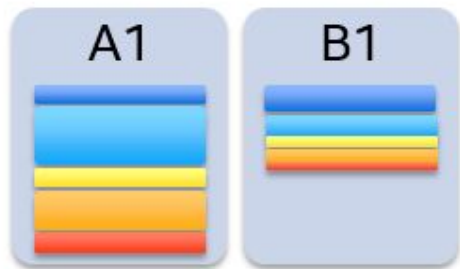


Sort Merge Join





Map output on Node 1



Map output on Node 2

Shuffle Join

Broadcast Join



Reduce tasks on Node 1



Reduce tasks on Node 2

Remote Shuffle Read



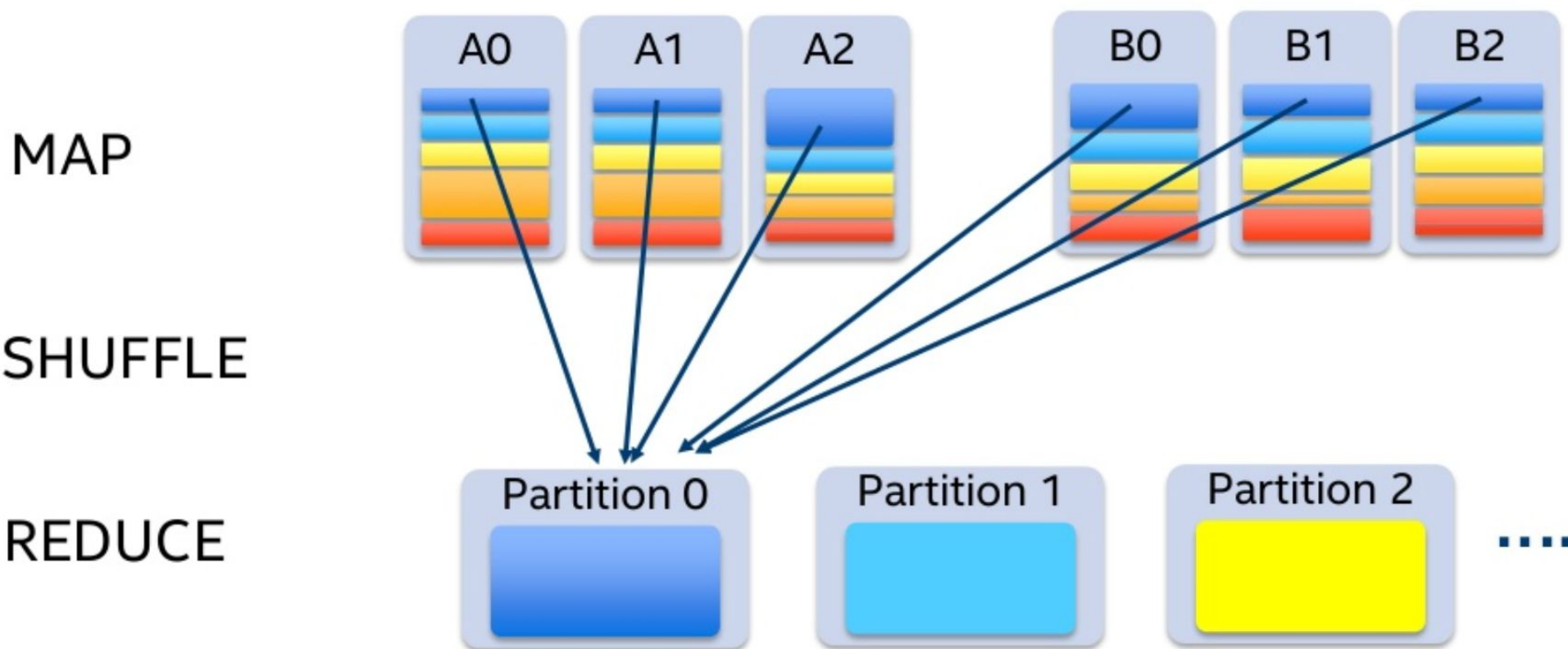
Reduce tasks on Node 1



Reduce tasks on Node 2

Local Shuffle Read

Shuffle Hash Join / Sort Merge Join



Sort-merge join algorithm explained

Customers table

Id	Login
2	User#2
1	User#1
4	User#4
3	User#3

Orders table

Id	User id
1001	2
1002	4
1003	4
1004	1

Sorting

Id	Login
1	User#1
2	User#2
3	User#3
4	User#4

Id	User id
1004	1
1001	2
1002	4
1003	4

Merging

1. Takes the first row in the left - (1, User#1). Does it have a match in the right ? Yes
-> **merge** (1, User#1, 1004, 1)
2. Takes the second row in the right - does it match (1, User#1) ? No
3. Gets the next row in the left - (2, User#2). Does it match the next value from the right ? Yes
-> **merge** (2, User#2, 1001, 2)
4. Checks again the next value in the right (1002, 4). Does it match (2, User#2) ? No
5. Goes to the left table and selects the next row (3, User#3). Does it match (1002, 4) from the right table ? No
6. Gets the next row from the left table (4, User#4). Does it match (1002, 4) ? Yes
-> **merge** (4, User#4, 1002, 4)
7. Gets the next row from the right table (1003, 4). Does it still match the current row from the left table ? Yes
-> **merge** (4, User#4, 1003, 4)
8. There are no more rows in both tables, we can finish the merge

Generated output: [(1, User#1, 1004, 1), (2, User#2, 1001, 2), (4, User#4, 1002, 4), (4, User#4, 1003, 4)]

Bucket Sort Example

- $A = (0.5, 0.1, 0.3, 0.4, 0.3, 0.2, 0.1, 0.1, 0.5, 0.4, 0.5)$

↓

bucket in array	
$B[1]$	0.1, 0.1, 0.1
$B[2]$	0.2
$B[3]$	0.3, 0.3
$B[4]$	0.4, 0.4
$B[5]$	0.5, 0.5, 0.5



Sorted list:

0.1, 0.1, 0.1, 0.2, 0.3, 0.3,
0.4, 0.4, 0.5, 0.5, 0.5